



Evaluating Code Quality of AI-generated Mobile Applications

A Comparative Study of React Native and Kotlin Implementations

Axel Jönsson
Nathalia Wehbi

This thesis is submitted to the Faculty of Faculty at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering. The thesis is equivalent to Weeks weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:

Author(s):

Axel Jönsson

E-mail: axjo21@student.bth.se

Nathalia Wehbi

E-mail: nawe22@student.bth.se

University advisor:

Lecturer Usman Nasir

Department of Software Engineering

Faculty of Faculty
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

The increasing integration of AI-powered tools in software development raises crucial questions about the quality of the code they generate, particularly in rapidly evolving fields like mobile application development. This study addresses the need for up-to-date evaluations of AI-generated code quality in non-native applications, a gap in current research.

To investigate this problem, we conducted an experiment where five prominent AI code generation tools – Gemini Code Assist, GitHub Copilot, ChatGPT, Windsurf IDE, and Deepseek – were prompted to generate code for a chess game in two mobile development frameworks: React Native and Kotlin. This resulted in a comparative analysis of ten AI-generated applications. The quality of the generated code was assessed using software quality metrics, informed by a comprehensive literature review.

Our analysis revealed a moderate to high degree of variation across the generated applications in key metrics such as cyclomatic complexity, lines of code, and cognitive complexity. However, the observed results did not provide conclusive evidence to definitively identify a single AI tool as consistently producing the highest quality code across both frameworks.

While the study provides valuable insights into the variability of code quality among different AI tools, the findings suggest that further research is necessary to achieve a more comprehensive understanding of the factors influencing the quality of AI-generated code. More in-depth investigation is required to draw definitive conclusions regarding the optimal AI tools for specific development contexts and to explore strategies for consistently generating high-quality code with AI assistance.

Keywords: AI-generated code, AI-assisted development, Large Language Models in coding, Code quality, Native and Non-Native applications.

Acknowledgments

We would like to give a special thanks to our supervisor, Usman Nasir, whose guidance, support and encouragement aided us greatly during the development of this thesis.

Contents

Abstract	i
Acknowledgments	iii
1 Introduction	1
1.1 Background	1
1.2 Research Aim and Objective	2
1.2.1 Aim	2
1.2.2 Research Questions:	2
1.2.3 Scope	3
1.2.4 Ethical, societal and sustainability aspects	3
1.3 Thesis Layout	3
2 Literature review	5
2.1 Mobile application development	5
2.1.1 Mobile operating systems	5
2.1.2 Developing mobile applications	6
2.2 Large Language Models	7
2.2.1 ChatGPT	9
2.2.2 Github Copilot	9
2.2.3 Deepseek	10
2.2.4 Gemini Code Assist	10
2.2.5 Windsurf (Codium)	10
2.3 Quality Metrics	11
2.4 Mapping literature to RQs:	13
3 Methodology	17
3.1 Research process	17
3.2 Method	18
3.2.1 Experiment Execution and Data Gathering:	18
3.2.2 Controlled Experiment and Variables:	19
3.2.3 Data Processing and Analysis:	20
3.3 Prompt Engineering:	20
4 Results and Analysis	25
4.1 Results	25

5	Discussion	31
5.1	Discussion	31
5.1.1	Limitations	32
6	Conclusions and Future Work	35
6.1	Conclusions	35
6.2	Future work	36
	References	37
A	Supplemental Information	41

1.1 Background

With the increasing use of AI-powered tools in software development, developers are hard pressed to keep up with the new methods of writing and optimizing code [21, 29].

Generative AI tools such as ChatGPT and Gemini Code Assist support programmers by generating code snippets, automating repetitive tasks and even building entire applications [3, 17, 31]. Although these tools offer great productivity benefits and support, the question remains about the quality of AI-generated code [11, 16, 19, 28, 30].

The existing studies that relate to code quality of AI-generated code is limited, especially ones that are up-to-date, since it is a rapidly evolving field with AI getting better and better at not only solving tasks, but doing so optimally [8, 17, 21, 29]. This study aims to address this gap by evaluating the quality of AI-generated code in Native and Non-Native applications.

AI-assisted development relies on Large Language Models (LLMs), which are trained on vast amounts of code to generate, complete and optimize programming tasks [1, 2, 17, 22, 29]. Models like OpenAI's GPT-4o assist developers to quickly producing functional code [3, 18, 30, 31].

In mobile development, applications can be built as native and non-native. Native applications are specifically designed for a particular platform, such as IOS (Swift) and Android (Kotlin), often resulting in better performance. Non-native applications use cross-platform frameworks such as React Native, Flutter, etc. This type of development allows developers to write code once and deploy it across multiple platforms. While non-native solutions offer flexibility and effectiveness they may introduce performance trade-offs [5].

The quality of AI-generated code is a critical factor of software development impacting maintainability, readability, and long-term sustainability [11, 21, 28, 30]. While AI-generated code can accelerate development, it may lack contextual understanding, leading to excessive structures, unnecessary complexity or poor solutions, unlike human-written code [11, 16]. Ensuring in high-quality AI-generated code it requires validation, refactoring and evaluation using established software engineering principles [19, 28]. We'll assess AI-generated code quality using established metrics to determine its maintainability, complexity and effectiveness. By analyzing these factors, we aim to gain insights into how AI impacts the maintainability of mobile applications.

1.2 Research Aim and Objective

1.2.1 Aim

By examining the potential impact of AI-generated code, this research aims to guide developers in making informed decisions when selecting AI tools and to analyze which tool best suits their specific application needs and requirements. Furthermore, by investigating how AI influences application complexity, this study contributes to the ongoing discussions about the role of AI in software engineering and its potential to optimize mobile application development. Through conducting experiments and reviewing existing literature, we seek to provide deeper insights into how AI-generated code affects the overall efficiency of mobile applications, ultimately assisting both individual developers and organizations in making well-informed decisions regarding the integration of AI-assisted coding into their development practices.

1.2.2 Research Questions:

1. *How does the code quality of AI-generated code differ between various LLMs?*
 - **Motivation:** Different large language models (LLMs) are trained on diverse datasets and employ different decoding strategies. These differences can lead to significant variations in the qualities of the code they generate. Understanding these differences in code quality (efficiency, maintainability) is crucial for developers and organizations to make informed decisions about which LLMs are most suitable for specific code generation tasks. Identifying the strengths and weaknesses of different LLMs in generating code can optimize development workflows and mitigate potential risks associated with using AI-generated code.
2. *What are the key factors that impacts the quality of AI-generated code?*
 - **Motivation:** Identifying the factors that influence the quality of code produced by LLMs is essential for improving the reliability and effectiveness of AI-assisted software development. Understanding these factors can guide the development of better prompting techniques, the design of more effective AI coding tools, and establish the best practices for using LLMs in software engineering. This knowledge can also help in identifying the limitations of current LLMs and areas where further research and development are needed.
3. *Which framework, React Native or Kotlin, results in better code quality when generated by LLMs?*
 - **Motivation:** React Native and Kotlin represent distinct approaches in mobile application development: React Native is a cross-platform framework, while Kotlin is the primary language for native Android development. LLMs may exhibit differences in ability when generating code for

these different environments due to their differences in training data related to each ecosystem, such as their syntactic complexities in the languages and architectural patterns that are commonly used in each of the framework's. Understanding these differences is crucial for developers choosing between native and cross-platform development when utilizing AI code generation tools.

1.2.3 Scope

This thesis examines and evaluates the quality attributes of AI-generated gaming applications created for both native and non-native mobile platforms [3, 5, 11, 16, 19, 28, 30]. The growing incorporation of artificial intelligence (AI) tools in the software development process requires a thorough understanding of their effects on the quality of the end product [11, 21, 29]. Therefore, this research directly addresses the implications of AI-generated code for various stakeholders within the context of mobile applications. A detailed understanding of the variations in AI-generated code quality across different mobile development paradigms is critical for assuring the quality of AI-assisted development [5, 19, 28, 30].

1.2.4 Ethical, societal and sustainability aspects

This study does not involve human subjects or personal information, reducing ethical concerns. However, the increasing use of AI is a controversial topic since many believe that jobs are in danger. This is a valid concern to have, however at the current stage, AI is not able to completely replace developers and they instead serve as an assistant and not as a replacement for humans. Further research need to be made to get a more detailed understanding of possible insidious side effects that AI-generated code may have and if AI is for example able to refactor existing code in an effective way.

Using AI-generated code may promote sustainability by streamlining development processes and increasing job satisfaction as well as well-being among developers. [21]

The human cost of training LLMs should not be overlooked. Larger LLMs are trained on vast amounts of data with billions of parameters. [20] In order for the generated output to be safe, non-toxic and not contain any illegal content, preemptive measures need to be taken. One way of achieving this has been to employ humans that are tasked with filtering out this toxic content. This can have a serious toll on the mental health of the people whose job it is to look at illegal, unsafe and quite horrible content for hours at a time. [4]

1.3 Thesis Layout

- **Chapter 1 - Introduction:**

- *Background:* Provides an overview of the research topic and explains the background information for the chosen study. As well as outlining the problems and questions.

- *Research Aim and Objective*: Clarifies the aim and objectives of the study. As well as the research questions and ethical concerns.
- *Thesis Layout*: Defines the layout of the thesis.
- **Chapter 2 - Literature review**: Reviews existing studies and researchers related to the topic, highlighting crucial information relevant to our study, and identifies research gaps.
- **Chapter 3 - Methodology**:
 - *Research Process*: Outlines the process of this combined research and experiment study.
 - *Method*: Explains the method used for the experiment.
 - * *Experiment Execution and Data Gathering*: Explains how the experiment is executed and what data is gathered from it.
 - * *Controlled Experiment and Variables*: Details the independent and dependent variables, while providing information on how to ensure a fair comparison.
 - * *Data Processing and Analysis*: Explains how the data is analyzed.
 - *Prompt Engineering*: Provides information on how prompting can influence the result and what criteria to follow to create a good prompt. As well as explain the used prompt in the experiment.
- **Chapter 4 - Results and Analysis**: Presents and summarizes the findings of the experiment.
- **Chapter 5 - Discussion**: Interprets the results in relation to the research questions and literature.
 - *Limitations*: Discusses the limitations related to our study.
- **Chapter 7 - Conclusion and Future Work**:
 - *Conclusion*: Summarizes the main findings from the results and discussion.
 - *Future work*: Provides information on how to expand the study in the future.

A literature review was conducted to establish the current state of research concerning AI-generated code quality, particularly within the domain of mobile application development. The review focused on peer-reviewed articles published in reputable journals and conference proceedings, primarily in IEEE Xplorer and Google Scholar.

The search strategy employs the following Boolean search string to identify relevant publications: ("AI-generated code" OR "AI-assisted code development") AND ("performance" OR "load time" OR "quality metrics") AND ("native" OR "React Native" OR "mobile").

The inclusion criteria for articles follows:

1. Publication within the last five years to ensure the relevance of contemporary AI models and software development practices.
2. Empirical studies employing robust evaluation methods, such as benchmarking and unit testing, to assess software performance and quality.
3. Coverage of relevant quality attributes such as code complexity, maintainability, etc.

2.1 Mobile application development

2.1.1 Mobile operating systems

Android is a significant platform in the development of mobile applications. Böhm [5], explored the cross-platform mobile application development and used Android native (Kotlin) as a baseline comparison to IOS. Their study highlighted that Android exhibited the best performance among the platforms studied. This underscores the efficiency and performance capabilities of the Android platform. The widespread popularity of Android is reflected in its status as the leading mobile operating system globally [5]. This extensive adoption can be primally attributed to its open-source framework, which allows manufacturers to customize the OS to fit the hardware configuration and price point. This open approach fosters a dynamic ecosystem, allowing developers to modify and adapt the operating system, implement features and distribute their applications through various channels.

In contrast, Apple’s iOS offers a more controlled and unified ecosystem. It’s known for its consistent performance, smooth user experience and strong emphasis on security. Their closed ecosystem provides less fragmentation, which may simplify the development process and ensure a more consistent user experience across devices. However, this also means that developers have less freedom to customize the OS and face stricter app review processes.

In conclusion, the Android operating system, with its open nature, diverse hardware ecosystem, comprehensive development tools, widespread adoption and the availability of robust measurement tools, makes it an ideal environment for in-depth analysis of quality characteristics in AI-produced code.

2.1.2 Developing mobile applications

Developing mobile application presents a vast amount of choices, each with its own set of trade-offs regarding performance, platform access and code reusability. Incorporating AI-assisted coding tools into this development, introduces a new discussion surrounding different approaches to mobile application development. The importance of understanding how the tools interact with these distinct approaches becomes crucial. By examining the fundamental differences and considering each approach, we can better investigate the potential impact of integrating AI into mobile application development workflows [3, 5, 21, 29].

Native mobile development involves building applications specifically for a single operating system such as Android or iOS, using the programming languages and tools provided by that platform’s ecosystem. Since they are developed using the platform’s core programming languages and API’s, native apps generally offer optimal performance, are highly responsive and well optimized for software features without intermediary layers [5]. Native applications also adhere to specific intuitive UI/UX guidelines of their respective platforms, providing a more integrated user experience. By using the native SDKs and programming languages, developers can create highly responsive and efficient applications tailored to the characteristics of each platform. While delivering great performance, native development often requires maintaining separate codebases for each platform, potentially increasing development time and cost [5].

Non-Native development, provides better code reusability by allowing developers to write code once and deploy it across multiple platforms. These applications embed these web technologies within a native container, leading to faster development and lower cost by sharing a significant portion of the codebase. However, non-native apps may experience performance trade-offs compared to native apps due to the need for bridges such as API’s to communicate with the underlying native system. Additionally, access to certain advanced native features might be limited or require relying on third-party plugins or custom native code [5].

Near native development with frameworks like React Native, Flutter and Kotlin Multiplatform (KMP) aim to bridge the gap between cross-platform code sharing and native performance. They achieve better performance than traditional hybrid apps through different architectural approaches [5].

While near-native development offers an interesting middle ground, our research will primarily focus on evaluating AI-generated code within purely native and non-

native mobile application development. This decision is driven by the desire to establish a clear contrast of code quality in both spectrums. Native development represents the benchmark for optimal platform integration and performance, while non-native development highlights the trade-offs associated with code reusability. By focusing on these two, we aim to provide a more direct and insightful comparison on AI-generated code quality implications under these approaches. This comparison will allow us to better understand the potential benefits and drawbacks of using AI-generated code in scenarios prioritizing whether platform-specific optimization or cross-platform efficiency.

2.2 Large Language Models

We aim to investigate models that are widely accessible to developers and represent the current state-of-the-art in AI-assisted code generation. By focusing on models with existing documentation and community support, we can ensure the relevance of our findings for other developers.

Furthermore, considering AI tools with different architectural approaches and training data, we can gain a broader understanding of how various AI technologies impact the code quality of mobile applications. This selection allows us to provide insightful knowledge for developers considering integrating AI tools into their mobile development workflows.

The differences between AI-tool and AI-model center around the LLM interaction with resources. The model is the LLM itself, it's the face of the AI you're interacting with. While the AI tool is the external entity that the LLM utilizes to extend its functionality. The tool provides the LLM access to capabilities such as retrieving information, executing code, or interacting with specific environments. Z. Wang [29] emphasizes that the LLM learns to call and integrate these external tools to achieve more complex tasks, highlighting the tool as a separate component that enhances the model's native abilities. Thus, the model is the central agent, and the tool is an external resource it learns to employ.

AI Tool	Description	Relevant Articles
ChatGPT	A large language model (LLM) capable of generating code and text.	[3], [31], [30]
GitHub Copilot	An AI pair programmer that provides code suggestions and auto completion.	[18], [30]
DeepSeek	A large language model focused on code generation and understanding.	Newer AI tools are therefore so recent that no comprehensive study is yet available.
Windsurf (Codeium)	An AI-powered code completion and search tool.	Newer AI tools are therefore so recent that no comprehensive study is yet available.
Gemini Code Assist	Google's AI tool for code generation and assistance.	Newer AI tools are therefore so recent that no comprehensive study is yet available.

Table 2.1: AI Tools

Table 2.1 provides an overview of the AI tools utilized and discussed. It details the primary function of each tool, how it is employed within the existing literature, specifically for generating code to be evaluated for quality and the articles that reference the tool. This summary emphasizes the role of LLMs in the code generation process and sets the stage for understanding their impact on software quality. The table includes several prominent AI code generation tools, such as ChatGPT, GitHub Copilot, DeepSeek, Windsurf, and Google's Gemini Code Assist.

Large Language Models (LLMs) are presented in these articles as a potentially disruptive force in the landscape of software development, especially in the context of code generation. Their ability to understand and generate human-like text has paved the way for their application in producing code.

Hou and Ji [17] lay the groundwork for understanding this potential. This research delves into the core ability of LLMs producing code that not only achieves functional requirements but also adheres to established coding standards and best practices. By directly contrasting LLM-generated code with that written by human programmers, this study aims to establish a baseline understanding of the current capabilities and limitations of LLMs as an assistive coding tool.

Building upon this, Coignon et al. [8] study offers a more targeted evaluation of the efficiency of LLM-generated code in a challenging context: competitive programming. LeetCode problems are famous for their often intricate algorithmic requirements and strict performance constraints. By analyzing how LLM-generated code performs in solving these problems, this research provides valuable information on the optimization capabilities of these models. The study examines metrics such

as execution time, memory usage, and the correctness of the solutions generated by LLMs, thereby offering a deeper understanding of their ability to produce not only functional but also effective code. This focused evaluation in a demanding environment highlights the potential of LLMs to assist even in complex coding tasks where efficiency is crucial.

2.2.1 ChatGPT

ChatGPT is a large language model (LLM) developed by OpenAI, often powered by their advanced GPT-4o model, recognized for its ability to process and generate human-like text, audio, and image outputs in real-time, including code [23]. Its capabilities are being examined for their influence in various software development contexts, including mobile application creation. Alwin [3] research provides a direct investigation into the practical utility of ChatGPT as a coding assistant, specifically for mobile development. This study explores a range of aspects, such as the types of mobile development tasks ChatGPT can effectively handle (generating UI components, implementing business logic, writing utility functions), the quality and correctness of the generated code, the level of human intervention required to integrate and debug this code, and the overall efficiency gains or challenges encountered by developers using ChatGPT in this domain.

Expanding on this, Zhang et al. [31] offer a broader perspective on the ability of ChatGPT to generate code. By evaluating its performance across a diverse set of programming languages relevant to software development, this study provides a comparative understanding of ChatGPT's strengths and limitations in different coding scenarios. The findings shows ChatGPT's ability in handling language-specific syntax, adhering to coding conventions, and the generalizability of its code generation capabilities. This multi-language analysis is crucial for understanding the specific contexts in which ChatGPT might be most effective as a mobile development tool and where its limitations might necessitate developer oversight or alternative approaches.

2.2.2 Github Copilot

GitHub Copilot functions as an AI pair programmer, offering real-time code suggestions and auto-completions directly within development environments. It is powered by generative AI models developed by GitHub, OpenAI, and Microsoft, trained on natural language text and source code from publicly available sources [12, 13]. It is the subject focus in Li et al. [18] study. This research delves into the practical ways in which Copilot assists developers during the coding process, primarily through real-time code suggestions and completions. The study's emphasis on "performance" suggests an evaluation not only of the functional correctness of the generated code snippets but also their efficiency in terms of execution speed and resource utilization. By examining Copilot's impact within a realistic coding workflow, the research aims to provide tangible insights into how AI-powered code suggestions affect developer productivity, the time required to complete coding tasks and the overall quality of the resulting software product. Given the widespread use of GitHub Copilot across various development domains, this study's findings are specially relevant for understanding the practical benefits and potential drawbacks of integrating AI assistants

into mobile development workflows.

2.2.3 Deepseek

DeepSeek is a large language model specifically designed with a strong focus on code generation and understanding. Its DeepSeek Coder series is trained from scratch on a massive source code database, including 87 percent code and 13 percent natural language in English and Chinese, enabling strong performance across various programming languages [9,10]. While not featured in a study solely dedicated to its mobile development applications within the research we chose, DeepSeek’s underlying architecture and training are considered in the broader evaluation of LLM performance. Tosi [28] research, which assessed multiple Large Language Model engines, including transformer-based models, provides a comparative perspective. Their conclusion that Models trained on domain-specific data, performed better in structural cohesion and logical consistency suggests a potential advantage in generating code that is better organized and more logically sound, at least in the specific contexts evaluated by Tosi. This finding implies that DeepSeek, with its training potentially focused on code and related technical domains, might exhibit strengths in producing code that adheres more closely to established programming principles compared to more general-purpose LLMs. While Tosi’s study might not be exclusively focused on mobile development nor DeepSeek, the identified strengths in structural cohesion and logical consistency could translate to benefits in generating more maintainable and understandable code.

2.2.4 Gemini Code Assist

Gemini Code Assist is Google’s AI-powered tool engineered to provide code generation and assistance to developers. It offers capabilities such as code completion, natural language chat, and code transformation, often leveraging the Gemini 1.5 Pro model for enhanced local codebase awareness [14,15]. Based on the set of research articles we included in this study, there is no specific study or analysis that directly investigates the capabilities or performance of Google’s Gemini Code Assist in the realm of code generation. Highlighting the gap of research into Gemini as a code assistant. Therefore, while Gemini Code Assist is a relevant AI-powered coding assistant in the broader landscape of AI in software development, its specific strengths, weaknesses, or suitability for mobile development, particularly in the context of generating or analyzing code is yet not explored.

2.2.5 Windsurf (Codium)

Windsurf (formerly Codeium) is an AI-powered code completion and search tool that aims to enhance developer productivity through features like code generation, analysis, and an advanced context engine that deeply understands the codebase [7]. Similarly, within the collection of research articles chosen, there is no specific study or evaluation dedicated to the Windsurf AI tool. While Windsurf offers features aimed at enhancing code quality and developer productivity through AI-powered test generation and code analysis, its performance and impact in the context of

generating or analyzing code for mobile applications, are not explored within these documents.

2.3 Quality Metrics

The quality of AI-generated code is a central focus of our research. Recognizing that functional correctness alone is insufficient for robust and sustainable software development, these studies delve into various aspects that contribute to the overall quality of AI-generated code. This section highlights the importance of evaluating not just whether AI can generate code, but also how well that code adheres to principles of good software engineering.

Fawareh et al. [11] directly tackle a crucial aspect of code quality by investigating the "impact of AI-generated code tools on software readability code quality factor." Their research examines how the use of AI in code generation affects the ease with which developers can understand and interpret the resulting code. Readability is paramount for collaboration, debugging, and future maintenance. The study's focus suggests an exploration of factors such as code clarity, the use of meaningful identifiers, code formatting and the presence of explanatory comments in AI-generated code.

Tosi [28] research, adopts a comparative approach to understanding the quality landscape of AI-generated code. By evaluating the output of various AI models or "engines" this study aims to identify potential differences in the quality characteristics produced by different AI technologies. This comparative assessment reveals which AI models excel in specific quality attributes and highlights potential strengths and weaknesses across the spectrum of available AI code generation tools.

Finally, He et al. [16] provide a fundamental benchmark for evaluating the quality of AI-generated code. By directly contrasting the AI-produced code with the code written by human developers, this study inherently assesses a wide range of quality attributes that are typically valued in software engineering. This comparison explores aspects such as correctness, efficiency, maintainability and adherence to established coding conventions. The insights gained from this comparative analysis are crucial for understanding how AI-generated code currently stacks up against human-authored code, as well as for identifying areas where AI tools need further improvement to meet the standards of professional software development.

Cyclomatic complexity, a widely adopted metric, quantifies the number of independent paths within a program's control flow graph. Measuring these paths serves as an indicator of code complexity and potential difficulties in testing and maintenance. Tosi [28] directly assessed this metric, finding that Cyclomatic complexity was generally higher in LLM-generated code, reflecting repetitious logic and insufficient abstraction. This suggests that AI-generated code may often contain more intricate and less streamlined control flow, potentially increasing the effort required for comprehension and modification.

Cognitive Complexity measures how difficult a given piece of code is for a human to understand and read. Various metrics are used to measure code diffi-

culty. It calculates the number of independent linear paths throughout the code. Cyclomatic Complexity quantifies how many distinct execution paths exist, while Cognitive Complexity evaluates how difficult it is for a developer to mentally trace and understand those paths [26]. This metric is however not mentioned in the resources we found, this is considered a gap in the research of this scope.

Lines of Code (LOC) is a fundamental size metric that counts the number of lines in a codebase. While a simple measure, it can often correlate with code complexity and maintainability; excessively long code segments can be harder to grasp and manage. According to Zhang et al. [31], ChatGPT produced more verbose code compared to humans, leading to inflated lines of code and reduced clarity. This tendency towards verbosity was also noted by He et al. [16], who observed increased code bloat in AI-generated samples. These findings suggest that AI-generated code may be longer than human-written code, potentially impacting readability and increasing the overall size of the codebase.

The Maintainability Rating is a composite metric designed to provide an overall indication of how easy software code is to understand and modify over time. This indicates that [18], while AI can generate code, the initial output may often fall short in terms of maintainability due to factors like excessive length and lack of consistent coding style. However, improvements with iterative prompting suggest that refining the interaction with AI tools can potentially lead to more maintainable code.

Technical debt refers to the long-term costs associated with maintaining a codebase, that has been implemented with shortcuts or poorer design choices to meet required needs. Yetiştirilen et al. [30] discuss how technical debt piles up in AI-generated code, where the rapid creation of AI code can neglect best practices or introduce complex structures that are hard to maintain. This metric is critical for understanding the sustainability of a project and predicting the effort required for future updates or modifications.

Issues of Bugs are critical indicators of code reliability and are often used to assess overall software quality. Melin et al. [19] analyzed bugs detected by SonarQube in Python programs generated by quantized large language models. Despite functional correctness, these AI-generated programs exhibited a higher incidence of bugs compared to human-written. The types of bugs identified included misused language features, inefficient logic, and problematic conditionals. These findings reveal that, while AI can produce working code, the underlying quality may suffer due to structural flaws or overlooked errors, requiring more harsh testing and review processes for real-world deployment.

The quality of AI-generated code is important for long-term health and sustainability. Key factors such as maintainability, complexity, and adherence to best practices play a crucial role in this context. While AI can speed up the development process, deficient code quality can lead to increased technical debt, elevated costs, and poor developer productivity over time. With the chosen metrics collected from this research process, our study seeks to provide information on AI's impact on both the user experience and long term maintainability of the codebase. Evaluating code generated by artificial intelligence is very important to understanding its practical

utility and potential impact on software development practices. This evaluation necessitates specialized tools that offer measurable data on both code quality, thereby identifying its strengths and weaknesses and guiding its effective integration into development workflows. The measurements tool chosen for this study is SonarQube.

SonarQube is a widely adopted platform for continuous inspection of code quality. It performs static analysis to detect bugs, code smells, security vulnerabilities and code coverage issues across a wide range of programming languages. SonarQube is a valuable tool for assessing the structural quality, complexity, and potential maintainability of code generated by AI tools [24].

Table 2.2, presents a summary of the software quality metrics used in this thesis, drawn from the reviewed research. It outlines each metric, its purpose, how it is measured, and the relevant articles that support its selection. This provides a concise overview of the criteria used to evaluate the quality of AI-generated code.

Metric	Result	Measurement	Relevant articles
Cyclomatic complexity	Potential difficulties in testing and maintenance.	Number of independent paths in the control flow graph.	[28]
Cognitive complexity	Measures other developers difficulty understanding the code.	Number of breaks in the linear flow of the code.	N/A
Lines of Code	Code size, which can correlate with complexity and maintainability.	Counts the total number of lines in the codebase.	[31], [16]
Maintainability Rating	Ease with which code can be understood and modified over time.	Composite score based on various factors such as complexity, LOC, etc.	[18]
Technical debt	Effort required to fix maintainability issues in the codebase.	Estimated in time (hours) or complexity.	[30]
Issues (bugs)	Errors affecting functionality and reliability	Numbers of bugs identified in code unit	[19]

Table 2.2: Quality Metrics

2.4 Mapping literature to RQs:

Table 2.3 below presents an overview of how each selected source contributes to answering the three research questions (RQs). The table highlights whether each study addresses differences in code quality between various large language models (RQ1), explores key factors influencing the quality of AI-generated code (RQ2), or provides insights specifically related to generating code for React Native and

Kotlin platforms (RQ3). This mapping helps clarify the scope and relevance of each source within the context of this research and identifies which works are most directly applicable to each question.

- **RQ1:** How does the code quality of AI-generated code differ between various LLMs?
- **RQ2:** What are the key factors that impacts the quality of AI-generated code?
- **RQ3:** Which framework, React Native or Kotlin, results in better code quality when generated by LLMs?

Ref	Title	RQ1: Code Quality Differences Between LLMs	RQ2: Key Factors Affecting Code Quality	RQ3: React Native vs Kotlin Generation
[1]	Using ChatGPT to Create Mobile Apps	No	Yes (usability, integration issues)	Yes (React Native focus)
[2]	AI-Assisted Coding: Kotlin vs React Native	Yes (indirect via comparison)	Yes (code reuse, performance)	Yes (direct comparison)
[3]	LLM-generated Code on LeetCode	Yes	Yes (problem complexity, prompt design)	No
[4]	Readability and Code Quality Factors	No	Yes (structure, logic)	No
[6]	AI vs Human Code: Comparative Study	Yes	Yes (semantics, correctness)	No
[7]	LLMs vs Human Programmers	Yes (multiple LLMs tested)	Yes (training data, prompt design)	No
[8]	Evaluation of AlphaCode	Yes (vs GPT-style LLMs)	Yes (competitive environment)	No
[9]	GitHub Copilot Case Study	Yes	Yes (IDE integration, feedback loop)	No
[11]	Sustainable Dev w/ GenAI	No	Yes (sustainability, team workflow)	No
[12]	Using AI in Real Projects	No	Yes (real-world usability)	No
[13]	Comparing AI Code Generators	Yes (GPT-4 vs Copilot etc.)	Yes (syntax vs logic abstraction)	No
[14]	ChatGPT Code Across Languages	Yes	Yes (language-specific issues)	Yes (includes React Native)

Table 2.3: Relevance of sources to research questions

This chapter lays the groundwork for evaluating the code quality of AI-generated mobile applications by first describing the mobile development landscape through an overview of Android and iOS, along with their respective strengths. Then discussing native, non-native, and near-native development approaches, underscoring the study's focus on comparing AI-generated code within native and non-native frameworks. The chapter further explores the potential of Large Language Models in code generation, referencing existing research and identifying a gap in the evaluation of newer models. Finally, it dedicates significant attention to defining key code quality metrics and introduces SonarQube as the tool employed to measure and assess these attributes in the AI-generated code.

3.1 Research process

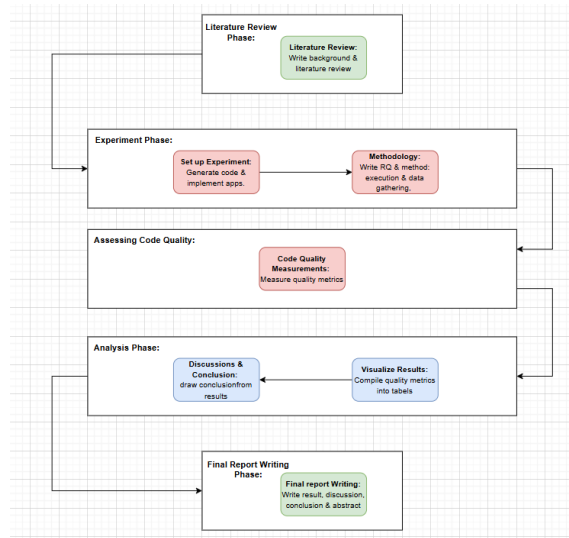


Figure 3.1: Process Diagram

The research process shown in as seen in Figure 3.1 above, included a multiphase process that combined the literature review with an experiment on AI-generated code.

Phase 1: involved identifying, selecting and analyzing existing research on AI code generation techniques, mobile development approaches and code quality. The literature provided a foundation for defining the research questions and determining the appropriate metrics and development approaches.

Phase 2: Following the review, a controlled experiment was set up and executed with a prompt tailored for the frameworks and LLMs, including role prompting, specifications and guidance. The codes were generated by multiple LLMs using both Native and non-Native approaches, for a standardized chess mobile application.

Phase 3: The games were assessed using SonarQube identifying code quality, forming the empirical data for addressing RQ 1 and 3.

Phase 4: The collected data from the experiment were assembled in tables visualizing the differences in the codebases. By analyzing the data, we could connect the behavior, prompt strategy and code quality outcomes. We drew insights from both the literature and our experimental results to create the final research report.

Drawing conclusions from the metrics, addressing the three research questions and identifying limitations and future research.

3.2 Method

This study aimed to evaluate the quality of generated code by various AI tools when developing mobile applications for both native and non-native environments. To achieve this, a controlled experiment study was conducted, focusing on the creation of chess games using five different AI-powered code generation tools. This approach was chosen for its ability to establish potential causal relationships between the independent variables (AI tool and development framework) and the dependent variables (quality metrics) through controlled manipulation and systematic measurement. Unlike observational studies or surveys, an experiment allows for a more direct assessment of the impact of AI-generated code on application characteristics under specific conditions.

The rationale for choosing a chess game was its moderate complexity, which required both logical implementation of game rules and interactive user interface elements, thus providing a subtle benchmark for assessing both functional correctness and code quality of AI-generated code. This choice allows for a focused evaluation of AI capabilities in handling complex logic and dynamic UI, making the insights gained broadly relevant to other software development challenges that demand similar levels of complexity in logic, state management, and user interaction. Although the study focuses on a chess application, the findings offer valuable perspectives on the general efficacy of AI in generating functional code for moderately complex interactive systems.

We selected five prominent AI code generation tools: ChatGPT, Github Copilot, DeepSeek, Gemini Code Assist and Windsurf. These tools were chosen based on their widespread adoption within the developer community and their diverse approaches to code generation, ranging from conversational AI to context-aware suggestions within IDEs.

The experiment involved generating a total of ten chess games, five implemented using native Android development principles with Kotlin and five implemented using non-native framework, specifically React Native. The decision to focus on native Android and React Native was driven by their significant presence in the mobile development landscape and the well-documented trade-offs between them in terms of performance and platform-specific feature access. This comparison allowed us to assess whether AI assistance influences these established trade-offs. We did not explicitly focus on "near-native" frameworks like Flutter or Xamarin in this initial study to maintain a clear distinction between fully native performance characteristics and the abstractions introduced by fundamentally cross-platform solutions.

3.2.1 Experiment Execution and Data Gathering:

The generated code for each of the ten chess games was implemented and executed within a controlled program environment. For native Android games, Android Studio was used with appropriate emulators. For React Native games, a suitable environ-

ment was set up that supports React Native development and execution on Android emulators.

Code quality was assessed using SonarQube, a static code analysis tool. SonarQube was configured to analyze the generated codebases for both native and non-native games, providing quantitative data on the following metrics:

- **Cyclomatic Complexity:** Measured the number of independent paths in the code.
- **Cognitive Complexity:** Measure of how difficult the application is to understand.
- **Lines of Code (LOC):** Counted the total number of lines in the codebase.
- **Maintainability Rating:** Provided a composite score indicating the ease of maintaining the code.
- **Technical debt:** Gather the estimated cost of delayed maintenance, which can lead to future challenges in maintenance and updates.
- **Issues (bugs):** Detected flaws in logic, structure, or syntax that can lead to malfunctioning or unreliable software behavior.

3.2.2 Controlled Experiment and Variables:

This study was designed as a controlled experiment with the following variables:

- Independent variables:
 - AI Code Generation Tools: (*ChatGPT, GitHub Copilot, DeepSeek, Gemini Code Assist, Windsurf*)
 - Development Frameworks: (*Kotlin, React Native*)
- Dependent variables:
 - Quality Metrics: *Cyclomatic complexity, Cognitive Complexity, Lines of Code, Maintainability Rating, Technical Debt and Issues (bugs)*.

The game logic and core requirements for the chess game were kept consistent across all ten implementations to minimize confounding variables and ensure a fair comparison. Prompt engineering was also standardized as much as possible across the different AI tools. A single prompt was initially used, but some AI tools needed more clarification and guidelines, which were included in smaller, additional prompts. However, it is acknowledged that the inherent nature of LLMs might still introduce some variability in the generated code even with controlled prompts.

3.2.3 Data Processing and Analysis:

The code quality metrics collected from SonarQube were compared across the different AI-generated games and development approaches. This analysis aimed to determine if there were discernible patterns or significant variations in quality based on the AI tool employed and whether the choice of native versus non-native framework affected with the AI's output.

By systematically generating, measuring, and analyzing the code quality of AI-generated chess games across native and non-native Android development, this methodology provides a structured approach to evaluating the capabilities and limitations of current AI tools in the context of mobile application development. The findings will contribute to a better understanding of how AI assistance impacts the trade-offs inherent in choosing between native and cross-platform development for Android.

3.3 Prompt Engineering:

Prompt Engineering is crucial in AI, especially for leveraging LLMs in code generation. Unlike traditional programming, which relies on deterministic commands, generating code with LLMs requires a sophisticated approach that accounts for their reliance on pattern recognition and statistics. A well crafted prompt, explaining the desired output is more than a simple instruction; effective prompts results in clearly defined function and behavior (clarity of intent), detailed technical specifications (requirement specificity), relevant context (contextual grounding) and defined code structure (structural guidance). This necessity for a well crafted prompt stems from a fundamental difference in how LLMs understand and generate code compared to human programmers. As OpenAI notes, LLMs lack genuine programming reasoning [22] meaning they operate by identifying statistical relationships in their training data. While enabling impressive code generation, this approach introduces limitations. Vague prompts can lead LLMs to latch onto irrelevant patterns, producing flawed code and since LLMs are sensitive to slight variation in prompt wording, which can lead to unexpected outcomes.

Research has developed advanced prompt engineering techniques to improve the quality and reliability of AI-generated code. Few-shot learning, as described in Googles Gemini API documentation [1] involves providing LLMs with input-output examples. Chain-of-thought prompting encourages step-by-step reasoning. Prompt decomposition breaks down complex tasks into simpler ones, as suggested by DeepSeek [2]. Role prompting illustrated in Windsurf's documentation [6] assigns a specific role to the LLM.

Recent research has also emphasized prompt engineering's importance in mobile app development. Alwin discussed its role in guiding LLMs to generate functional code for small mobile apps [3], noting that well constructed prompts are crucial for obtaining usable code and that prompt clarity and structure significantly affect output quality [3]. This is identified in ChatGPT's ability to reduce development time and create more compact programs compared to human-developed code, but also observed inconsistencies that may influence its adoption [3]. This highlights the

necessity of careful prompt engineering to ensure efficient and functional code.

Li et al. introduces the necessity of effective prompt engineering for generating high-quality code with GitHub Copilot [18]. They explained that generated code quality depends on prompt accuracy, detail and that specific prompting strategies can increase the accuracy and reliability of the code produced. Their research indicates that while Copilot can produce functionally correct code, it may exhibit performance regressions compared to human written code [18]. However, they also suggest that effective prompt design can mitigate these performance drops, demonstrating its potential to improve AI-generated code performance [18].

To ensure our prompt adheres to the principles of effective prompt engineering, we structured it as follows:

These were the two primary prompts that we used for each AI-tool. We adjusted the prompts to suit the frameworks by switching out React Native for Kotlin as well as including Jetpack Compose for the GUI when generating Kotlin chess games:

First prompt (React Native):

Hi, I want you to write code that is readable and not too complex. A junior developer should be able to understand it.
This is what i want you to do:
Create a full chess game in React Native without using external chessboard libraries. Implement logic for all chess rules, including legal moves of all pieces (castling, en passant, pawn promotion, and checkmate detection will come later, do not include this for now).

Second prompt (React Native):

The app should generate a fully interactive chessboard with drag-and-drop functionality for pieces. The board should update dynamically based on player moves, and piece movement should be validated programmatically. Use React Native components for rendering the board and pieces, ensuring a smooth and responsive UI.

First prompt (Kotlin):

Hi, I want you to write code that is readable and not too complex. A junior developer should be able to understand it.
This is what i want you to do: Create a full chess game in Kotlin using Jetpack Compose for the GUI. Do not use external chessboard libraries. Implement logic for all chess rules, including legal moves of all pieces (castling, en passant, pawn promotion, and checkmate detection will come later, do not include this for now)..

Second prompt (Kotlin):

The app should generate a fully interactive chessboard with drag-and-drop functionality for pieces. The board should update dynamically based on player moves, and piece movement should be validated programmatically.

The prompts we created aligns with the principles of effective prompting. We modified the prompt based on the documentations recommendations. ChatGPT suggested giving the AI a role, which we did by adding "you are a developer", thus providing context for the LLM. Deepseek's initial output was grouped into a single file. To avoid this, we added "Please divide the code up into multiple files." to the prompt we used for Deepseek, in order to get an output that could be compared more honestly to the other AIs outputs.

Informing the LLM to use specific programming languages "React Native/Kotlin" and the constraint of not using "external chessboard libraries". Stating the primary objective: "create a full chess game" also elaborates on the key functionalities, such as generating a "fully interactive chessboard" and implementing "drag-and-drop functionality". The prompt specifies the requirements by explaining the layout, design or technical solution, such as "React Native components for rendering", "smooth and responsive UI". As well as define the input and output results by stating that "the board should update dynamically based on player moves, and piece movement should be validated programmatically". By adhering to these principles, we achieved a well-defined prompt that effectively communicates the desired outcome to the LLM, increasing the likelihood of generating high-quality code that meets our requirements.

In some cases the second prompt was not needed in order to get a fully functional and executable chess game. Windsurf and Gemini's React Native versions only required the first prompt in order to achieve this. Their Kotlin versions did not need further prompts in order to get the base logic in place, however they had some problems with generating the necessary settings files needed for a Kotlin project.

Across all Kotlin applications, the content in build.gradle, gradle.properties, settings.gradle, Theme.kt, themes.xml, colors.xml and AndroidManifest.xml was initially missing. This needed to be solved by prompts such as these:

You forgot to add the build.gradle file content at the root level, as well as gradle.properties and settings.gradle at root level.
--

Could you add content to colors.xml and themes.xml? Also, don't I need Theme.kt?
--

I need some content in @AndroidManifest.xml

These files are not included in the analysis of the code quality, since they are not Kotlin files, but are still needed to execute the application. As you can see in the third example prompt given; tagging of files was applied to the prompts for the AIs that had access to the code base, thus following best practices. After the necessary Kotlin files were added, most of the following errors were related to the content of these files. On average it did not require much more than 2-3 prompts to fix any given issue with the actual code, and the number of bugs that prevented a fully functional game was limited. Other common issues and shortcomings of the initial output from the AIs from the first two prompts were that the movement logic was not fully functional, not being able to capture other pieces and not having turn-based movement logic. In some cases functions did not have any code within them, instead the AIs simply generated the functions shell and a comment describing what logic

should be placed there.

As mentioned, all code generated by the AI tools were implemented in a suitable environment, which means that **all errors that occurred during the process were corrected by us to make it runnable**. Through this process, we could also review our developer experience on integrating the AI-generated code.

4.1 Results

Having conducted our controlled experiment where we implemented 10 chess game applications and made note of their code quality metrics from SonarQube, the following section will attempt to give the reader an understanding of how the AI-tools scored in terms of code quality. Figure 4.1 illustrates an example of a chess game, taken as a screenshot from the Android Studio Emulator running Geminis Kotlin application. Similar appearances and functionality is common across all applications, although variations occur, such as the highlighting of possible moves missing from some applications.

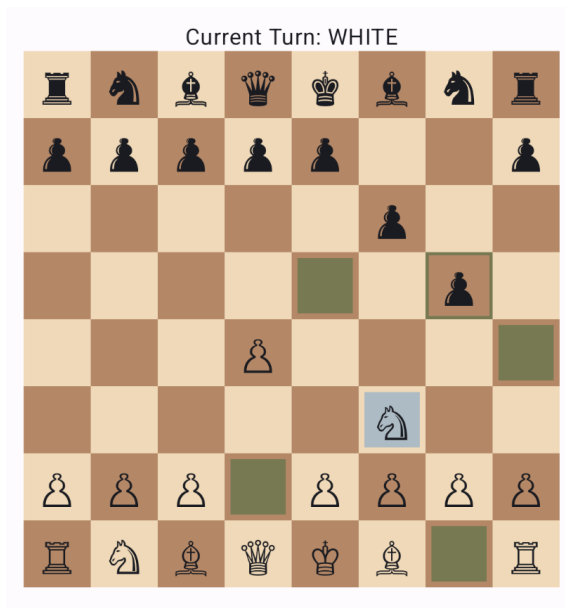


Figure 4.1: Geminis Kotlin chess game

Figures 4.2 and 4.3 visualize code snippets from Deepseeks Kotlin and React Native applications, where the movement logic for pawns take place. These snippets of code are of particular interest since Deepseeks quality metrics had relatively high values, such as it's cyclomatic complexity, which was the highest observed as described in Tables 4.1 4.2 4.3 The cyclomatic complexity of Deepseeks React Native movement logic file was 54, and it's Kotlin version had a cyclomatic complexity of

53. In contrast, both Windsurfs React Native and Kotlin Kotlin application had a cyclomatic complexity of 43 for the file with movement logic.

In Appendix A figures A.1 - A.7 the complete Kotlin and React Native files for Deepseeks movement logic are displayed.

```

49
50 const isValidPawnMove = (board, from, to, color) => {
51   const direction = color === COLORS.WHITE ? -1 : 1;
52   const startRow = color === COLORS.WHITE ? 6 : 1;
53   const rowDiff = to.row - from.row;
54   const colDiff = Math.abs(to.col - from.col);
55
56   // Moving forward
57   if (to.col === from.col) {
58     // Single move forward
59     if (rowDiff === direction && !board[to.row][to.col]) {
60       return true;
61     }
62     // Double move from starting position
63     if (from.row === startRow && rowDiff === 2 * direction &&
64         !board[from.row + direction][from.col] &&
65         !board[to.row][to.col]) {
66       return true;
67     }
68     return false;
69   }
70
71   // Capturing diagonally
72   if (colDiff === 1 && rowDiff === direction && board[to.row][to.col]) {
73     return true;
74   }
75
76   return false;
77 };
78

```

Figure 4.2: Deepseek React Native code

```

25
26 private fun getPawnMoves(
27   position: Position,
28   player: Player,
29   board: Map<Position, Piece>
30 ): Set<Position> {
31   val moves = mutableSetOf<Position>()
32   val direction = if (player == Player.WHITE) 1 else -1 // White moves up (row increases)
33   val startRow = if (player == Player.WHITE) 1 else 6
34
35   // 1. Move forward one square
36   val oneStep = Position(position.row + direction, position.col)
37   if (oneStep.isValid() && board[oneStep] == null) {
38     moves.add(oneStep)
39     // 2. Move forward two squares (only from starting row)
40     if (position.row == startRow) {
41       val twoSteps = Position(position.row + 2 * direction, position.col)
42       if (twoSteps.isValid() && board[twoSteps] == null) {
43         moves.add(twoSteps)
44       }
45     }
46   }
47
48   // 3. Capture diagonally
49   val captureOffsets = listOf(-1, 1) // Left and right columns
50   for (offset in captureOffsets) {
51     val capturePos = Position(position.row + direction, position.col + offset)
52     if (capturePos.isValid()) {
53       val pieceAtCapture = board[capturePos]
54       if (pieceAtCapture != null && pieceAtCapture.player != player) {
55         moves.add(capturePos)
56       }
57     }
58   }
59   // Note: En passant is not implemented here
60
61   return moves
62 }
63

```

Figure 4.3: Deepseek Kotlin code

The results from our controlled experiment, where we compare the code quality metrics of Kotlin and React Native applications generated by five different AI-tools are displayed in Table 4.1 and Table 4.2. Each table groups the React Native and Kotlin applications together into separate tables, where Cyclomatic Complexity (CC), Lines of Code (LoC), Maintainability Rating (MR), Cognitive Complexity, Technical Debt, and the number of Issues (bugs) are displayed, as identified by SonarQube. Table 4.3 below and figures A.8 - A.11 in Appendix A gives a simple to understand visualization of the metrics.

React Native Applications						
	CC	LoC	MR	Cognitive complexity	Technical debt	Issues (bugs)
Deepseek	110	407	A	78	1h 37min	19
Gemini	89	334	A	62	2h 6min	22
Windsurf	74	256	A	38	1h 21min	13
ChatGPT	93	242	A	49	1h 25min	12
Copilot	89	346	A	67	1h 33min	14

Table 4.1: Quality Metric Results for React Native applications

Kotlin Applications						
	CC	LoC	MR	Cognitive complexity	Technical debt	Issues (bugs)
Deepseek	93	392	A	73	1h 5min	10
Gemini	89	466	A	85	1h 9min	15
Windsurf	77	256	A	73	59min	11
ChatGPT	59	188	A	96	1h 31min	9
Copilot	63	269	A	56	1h 49min	29

Table 4.2: Quality Metric Results for Kotlin applications

GitHub Copilot’s Kotlin implementation exhibited the second lowest Cyclomatic Complexity and Lines of Code but the highest number of issues. When it comes to Copilots React Native implementation, we can see that it exhibits moderate values across most metrics. In contrast, Gemini’s Kotlin output showed the highest Lines of Code and Cognitive Complexity, while its React Native implementation also presented a high number of issues and significant technical debt. Windsurf generally produced code with lower complexity metrics in both frameworks, while Deepseek tended towards higher complexity and issue counts in React Native. ChatGPT demonstrated relatively low Lines of Code in both environments. The Technical Debt varied across the generated codebases, with Gemini’s React Native implementation showing the highest estimated debt. These initial results show significant

differences in the code quality attributes produced by different AI tools across the two mobile development frameworks.

Copilots and ChatGPTs Kotlin applications have the highest and lowest Issue count, respectively. The application with the highest score in Cyclomatic Complexity is the React Native Deepseek application, meanwhile, the one with the lowest score in Cyclomatic Complexity is ChatGPTs Kotlin application.

The number of Issues is also of interest, however the results above can be somewhat misleading, since an issue with a low severity rating counts the same as one with a high severity rating. Copilots Kotlin application is a rather extreme example of this, having only one issue which is of high severity while the rest are counted as low severity, mostly consisting of the "Unused import" warning. Another common type of issue raised by SonarQube for the React Native applications were "X is missing in props validation" for functions parameters. We did not use Typescript and so this issue can be forgiven, since it is partly caused by the nature of JavaScript. SonarQube also raised issues regarding nested ternary operations, suggesting they should be extracted into independent statements instead. Another issue raised commonly across the React Native applications was the use of array indexes in keys, which could cause the keys to not match up between renders, thus recreating the DOM, since the order of items in the array may change.

AI-tools average values						
	CC	LoC	Maintainability	Cognitive complexity	Technical debt	Issues (bugs)
Deepseek	101,5	399,5	A	75,5	81 min	14,5
Gemini	89	400	A	73,5	97,5 min	18,5
Windsurf	75,5	256	A	55,5	70 min	12
ChatGPT	76	215	A	72,5	88 min	10,5
Copilot	74,5	307,5	A	61,5	101 min	21,5

Table 4.3: Quality metric average values for AI-tools

Table 4.3 summarizes the average metric values for the different AI-tools. Deepseek has the highest cyclomatic complexity, cognitive complexity as well as the second highest LoC. It's maintainability rating is the same as the others, however its technical debt and issue count is quite moderate, falling somewhere in the middle.

Gemini has the second highest cyclomatic complexity, cognitive complexity, technical debt and issue count, as well as the highest LoC count.

Windsurfs LoC count, cognitive complexity, technical debt and issue count is among the best noted.

ChatGPT has the lowest issue count and LoC, with its other values being quite good as well.

Copilot has a relatively low cognitive complexity, however its issue count and technical debt is the highest among the average values.

Framework average values						
	CC	LoC	Maintainability	Cognitive complexity	Technical debt	Issues (bugs)
React Native	91	317	A	58.8	96.4 min	16
Kotlin	76.2	314.2	A	76.6	78.6 min	14.8

Table 4.4: Quality metric average values for frameworks

The React Native applications had on average a higher cyclomatic complexity, with a 14,8-point increase compared to the Kotlin applications average value. The lines of code were almost identical between the average values of React Native and Kotlin applications. The cognitive complexity of the React Native applications receives a better score, while the maintainability remains the same. The technical debt and issue count is similar between the React Native and Kotlin applications, however Kotlin has a slightly better score.

5.1 Discussion

One of the more interesting findings that we can see based on the results in table 4 is that despite there being quite a big variation in values among the applications, each application still rates an "A" in Maintainability. This could partly be due to the applications being small and not sufficiently complex to warrant a lower maintainability rating.

However, maintainability is hard to measure precisely, since it may be caused by multiple factors. The Maintainability Index was introduced as an attempt of measuring the maintainability of code, by leveraging metrics such as cyclomatic complexity, lines of code and Halstead's Volume. The maintainability index has some shortcomings itself, such as having an over reliance on lines of code and it being based on averages, causing it to be an inaccurate description of the actual maintainability. [27]

SonarQube has a rather simple solution to their maintainability rating, namely that it is per default solely based on the technical debt ratio. In this case a maintainability rating of "A" simply means that the technical debt ratio is below 5%. [25] This does tell us however that each AI managed to produce code with a quite good rating when it comes to technical debt in both Kotlin and React Native.

It is important to note that while the metrics discussed give us some insight into the quality of the code, they do not tell us anything about the experience of using the AI tools themselves. Nor does it give us any insight into the development of the applications and the degree of ease that the developer experienced when using the AI tools. **This is particularly interesting to note given that some of the applications exhibit results that can be considered good, meanwhile the experience when working with the AI-generated code and the AI-tool itself was not as positive, and vice versa.**

For example, Gemini's React Native application scores comparatively high in cyclomatic complexity, lines of code, cognitive complexity and has the highest score in technical debt as well as the second highest number of issues. However, implementing the AI-generated code that Gemini produced into a functioning application was among the most seamless experiences among all the applications.

Likewise, Copilot has across the board quite moderate values in most metrics, meanwhile it was the AI that was, from our experience, the most difficult to work with. The Copilot applications had to be redone from the ground up a number of times and they required quite a bit of work to get functioning. In the end, the Copilot

applications still did not meet the same requirements of functionality that the rest managed to achieve, namely the movement and capture of pieces was left unfinished.

Based on the average metric values for the different AI-tools, as shown in Table 4.3, it would seem that Windsurf has the most reliable results, with Deepseek and Gemini being the ones with the worst result. Although Deepseeks and Geminis average values were overall the highest, the difference between them and the rest were not that significant, except for Deepseek and Geminis cyclomatic complexity which had the average score of 101,5 and 89 respectively, with the others falling around 75.

The similarity in the frameworks average lines of code (317 for React Native and 314,2 for Kotlin), as seen in Table 4.4, suggests that the choice of framework does not play a critical role to the lines of code. However, when we observe the highest and lowest lines of code counts, Deepseeks React Native app with 407 LoC and ChatGPTs React Native app with 188 LoC, we see that there is a more than double increase between them.

However, the AIs seem to have more similar counts of LoC between their own React Native and Kotlin applications, with the exception of Gemini and Copilot.

These findings suggests that the lines of code can vary widely between AIs, as seen for example in Table 4.2, where ChatGPT has 188 lines of code and Gemini has 466. Lines of code as a metric is not sufficient to tell which AI produced the best result however, i.e less lines of code is not always preferable. It is however an indicator that the code generated by AIs is not guaranteed to be efficient when it comes to LoC, resulting in bloated code.

This assumption has to be put in its proper context. The applications themselves are chess games, with multiple files to accommodate the pieces, chessboard, movement logic and the game process itself. With this in consideration we can draw the conclusion that a LoC count of 466 (which is the highest count among all the applications) is not an unreasonable amount, if perhaps somewhat high.

Further research would be needed to gain deeper insight into how LoC of AI-generated code may vary when it comes to the complexity of prompts.

When looking at the metrics as a whole, we can make assumptions about the quality of the generated code that the various AIs produced. In this case it seems like Windsurf produced on average the most reliable results, as it never had the worst result and never fell behind third place in any metric. Furthermore it frequently recieved the best result, as seen in Tables 4.1 4.2 Meanwhile, other AI tools such as ChatGPT, which also had some of the better results exhibited greater variation and unreliability, such as in the React Native versions cyclomatic complexity, where it had the second highest score, as seen in Table 4.1

5.1.1 Limitations

It is important to be mindful of limitations regarding any potential insights and contributions to the wider body of research that this study aims to achieve. Below follows a non-comprehensive list of such limitations.

1. **Generality of results:** The findings are based on specific tasks, programming languages, performance and quality metrics. Results may vary for other tasks or

programming environments. Which won't generalize well to different industries or real-world production environments.

2. **Prompt sensitivity:** The quality and efficiency of AI-generated code are highly sensitive to the phrasing of prompts. Subtle changes in wording can lead to significantly different outputs as well as insufficient wording can lead to a poorer result. This could affect our end result. [20]
3. **Dependency on external Tools:** The results depend on external tools such as benchmarking frameworks, performance and quality measurement, which might have their own limitations. Since automated testing platforms don't stimulate real-world workloads, it might limit generalizability and lead to variations in results.
4. **Code correction:** The codes generated by our chosen AI generators will be implemented and compiled in a program environment. Errors encountered during compilation were corrected, this represents as a limitation because the corrections might have influenced the final product.
5. **Rapid evolution of AI models:** AI models like ChatGPT and GitHub Copilot are continuously updated, which may lead to different results over time. Meaning the quality results we conduct may quickly become outdated.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

The central insight that we have gained from our investigation into code quality of AI-generated mobile application code is that there can be a great variation in the quality metrics when comparing multiple applications with each other. Given that we observe such unpredictable variation in the quality metrics, we are unable to give definitive answers to some of our research questions, due to our sample size not being large enough to tell if our results are representative of the average output of these AI-tools.

Regarding our first research question, which concerns itself with the actual code quality and how it differs between various LLMs, we found that there does not seem to be a clear winner in terms of an LLM that is able to get the best results across the board. In most cases, the AIs produce acceptable results, but the generated code may exhibit unpredictable and rather substantial variation for various metrics. However, Windsurfs high reasoning o3 model gave results that were some of the best, exhibiting quite reliable output. Although it did not have the absolute best results across every metric, it had in most cases at least the second best result and were never far behind in the few cases that it did not have either first or second best place.

When it comes to the second research question, which examined factors that have a key impact in the quality of AI-generated code, we found that in order to ensure that the AI-generated code is as good as possible, it is important that a well crafted prompt is used. It is also important to make use of the AI-tool to its fullest extent, by utilizing features such as Windsurfs Cascade function or the file tagging feature in Gemini Code Assist and GitHub Copilot, used to let the AI know which files are relevant. Breaking larger tasks up into multiple prompts is a recurring recommendation found in most AI-tools documentation.

Given the relatively small sample size of our data, we are unable to give a definitive answer to the third research question. However, it does seem like the LLMs are able to generate code that receives similar scores in most metrics. React Native seems to have a higher cyclomatic complexity, meanwhile Kotlin has a higher cognitive complexity but other than that the frameworks receive similar scores across the metrics. To fully answer this question you would have to take the inherent differences between the frameworks into account, in order to differentiate between the effect that the frameworks and AIs have. It could for example be the case that Kotlin has on average a higher LoC than React Native, in which case it would not be an unexpected result to see this reflected in the metric and it might not be caused by

the AI that you are analyzing.

6.2 Future work

To further build on this study, several areas could be considered. For example, examine the quality of AI-generated asynchronous operations, state management solutions or UI rendering logic in both frameworks. Furthermore, exploring the impact of different prompting strategies and the level of detail provided in prompts on the resulting code quality. Investigating the evolution of AI-generated code quality over time, as LLM architectures and training data continue to advance, is also crucial for understanding the long-term implications for software development practices. As well as further broadening the quality aspect of the code to include metrics, such as security vulnerabilities and performance benchmarks. During the literature search of this study, we identified a lack of research papers that looked at the performance of AI-generated code, this could be an interesting avenue to pursue. Coignion et al. [18] look at the performance of solutions generated by LLMs for LeetCode problems. Using this format as a prototype, a possible combination of our study with theirs would be to look at the performance of AI-generated code for mobile applications. A study such as this would provide many valuable insights, not least regarding the ability of AI-tools to produce code with good performance when prompted to generate entire applications.

Incorporating developer feedback on the usability and maintainability of AI-generated code, would provide a more comprehensive understanding of the true impact of these tools on mobile application development. Furthermore, conducting a more comprehensive analysis on AI-generated mobile-application codes maintainability would counteract some of the limitations of this study regarding the insights we were able to attain by only using SonarQube as our source.

User experience of AI-tools is a relevant topic to explore further. This study found that the developers subjective experience of using the various AI-tools did not always match the AI-tools produced code quality. Conducting research which combines AI-tools code quality and user experience would provide more comprehensive insight into the validity of incorporating AI into developer workflows.

References

- [1] G. AI, “Prompting strategies,” <https://ai.google.dev/gemini-api/docs/prompting-strategies>, 2024, accessed: 2025-05-09.
- [2] T. AI, “Prompting strategies,” <https://docs.together.ai/docs/prompting-deepseek-r1>, 2024, accessed: 2025-05-09.
- [3] E. Alwin, “Using chatgpt to create mobile apps: Analysis of using chatgpt as a programming tool for developing smaller mobile applications.” DIVA: Digitala Vetenskapliga Arkivet, 2024. [Online]. Available: <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1865437&dswid=4678>
- [4] Billy Perrigo, “Exclusive: Openai used kenyan workers on less than \$2 per hour to make chatgpt less toxic,” <https://time.com/6247678/openai-chatgpt-kenya-workers/>, 2023.
- [5] J. Böhm, *AI-Assisted Coding and Comparative Performance Analysis of Cross-Platform Mobile Applications: Kotlin Multiplatform vs. React Native*. Universität Hamburg, 2024. [Online]. Available: https://edoc.sub.uni-hamburg.de/informatik/volltexte/2024/274/pdf/Masterarbeit_Boehm_Jacqueline.pdf
- [6] Codeium, “Prompt engineering best practices,” <https://docs.codeium.com/best-practices/prompt-engineering>, 2024, accessed: 2025-05-09.
- [7] Codeium, “Overview - codeium docs - windsurf,” <https://docs.codeium.com/context-awareness/overview>, n.d.
- [8] T. Coignon, C. Quinton, and R. Rouvoy, “A performance study of llm-generated code on leetcode,” in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 79–89. [Online]. Available: <https://doi.org/10.1145/3661167.3661221>
- [9] DataScientest, “Deepseek coder v2: The ai that revolutionizes developers’ daily lives,” <https://datascientest.com/en/all-about-deepseek-coder-v2>, mar 2024,
- [10] DeepSeek AI, “Deepseek coder: Let the code write itself,” <https://deepseekcoder.github.io/>, n.d.
- [11] H. Fawareh, H. M. Al-Shdaifat, A.-R. M, F. A. Fawareh, and M. Khouj, “Investigates the impact of ai-generated code tools on software readability code quality factor,” in *2024 25th International Arab Conference on Information Technology (ACIT)*, 2024, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/10876897>

- [12] GitHub, “Ai models for copilot,” <https://docs.github.com/en/copilot/using-github-copilot/ai-models>, n.d.
- [13] —, “Github copilot – your ai pair programmer,” <https://github.com/features/copilot>, n.d.
- [14] Google Cloud, “Gemini code assist: an ai coding assistant,” <https://cloud.google.com/products/gemini/code-assist>, n.d.
- [15] —, “Gemini code assist standard and enterprise overview,” <https://cloud.google.com/gemini/docs/codeassist/overview>, n.d.
- [16] S. He, J. Xuan, and W. Shang, “A comparative analysis between ai generated code and human written code: A preliminary study,” in *2024 IEEE 32nd International Conference on Program Comprehension (ICPC)*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10825958/references#references>
- [17] W. Hou and Z. Ji, “Comparing large language models and human programmers for generating programming code,” *arXiv e-prints*, p. arXiv:2403.00894, Mar. 2024. [Online]. Available: <https://advanced.onlinelibrary.wiley.com/doi/full/10.1002/advs.202412279>
- [18] S. Li, Y. Cheng, J. Chen, J. Xuan, S. He, and W. Shang, “Assessing the performance of ai-generated code: A case study on github copilot,” in *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, 2024, pp. 216–227. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10771230>
- [19] E. L. Melin, A. J. Torek, N. U. Eisty, and C. Kennington, “Precision or peril: Evaluating code quality from quantized large language models,” *arXiv preprint arXiv:2411.10656*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.10656>
- [20] Miguel Carreira Neves, “Advanced prompt engineering - practical examples,” <https://www.tensorops.ai/post/prompt-engineering-techniques-practical-guide>, 2023.
- [21] Murnawan, M. R. A. Ghazali, and V. Azzahra, “Empowering sustainable software development with generative ai,” in *Proceedings of the Widyatama International Conference on Engineering 2024 (WICOENG 2024)*. Atlantis Press, 2024, pp. 128–138. [Online]. Available: https://doi.org/10.2991/978-94-6463-618-5_14
- [22] OpenAI, “Best practices for prompt engineering with openai api,” <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-openai-api>, 2023, accessed: 2025-05-09.
- [23] OpenAI, “Introducing 4o image generation,” <https://openai.com/index/introducing-4o-image-generation/>, 2024,
- [24] SonarSource, “Sonarqube server,” <https://docs.sonarsource.com/sonarqube-server/latest/>, 2025.

- [25] —, “Understanding measures and metrics,” <https://docs.sonarsource.com/sonarqube-server/latest/user-guide/code-metrics/metrics-definition/>, 2025, accessed: 2025-06-13.
- [26] —, “Understanding measures and metrics,” <https://docs.sonarsource.com/sonarqube-server/latest/user-guide/code-metrics/metrics-definition/>, n.d.
- [27] Tim Gilboy, “Maintainability index - what is it and where does it fall short?” <https://sourcery.ai/blog/maintainability-index>, 2022.
- [28] D. Tosi, “Studying the quality of source code generated by different ai generative engines: An empirical evaluation,” *Future Internet*, vol. 16, no. 6, 2024. [Online]. Available: <https://www.mdpi.com/1999-5903/16/6/188>
- [29] Z. Wang, Z. Cheng, H. Zhu, D. Fried, and G. Neubig, “What are tools anyway? a survey from the language model perspective,” *arXiv preprint arXiv:2403.15452*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.15452>
- [30] B. Yetiştiren, I. Özsoy, M. Ayerdem, and E. Tüzün, “Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt,” *arXiv preprint arXiv:2304.10778*, 2023. [Online]. Available: <https://arxiv.org/pdf/2304.10778>
- [31] W. Zhang, Y. Liu, H. Chen, and M. Li, “Analysis of chatgpt-generated codes across multiple programming languages,” in *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10870152>

Appendix A

Supplemental Information

GitHub repository for the applications used in this study:
<https://github.com/Axjo21/PA1445>

Grammarly usage:

To ensure clarity and grammatical accuracy in the final text. We used Grammarly (Grammarly, Inc.) to identify and correct errors in grammar, spelling and punctuation.

```
1  import { PIECE_TYPES, COLORS, LETTER_TO_PIECE, LETTER_TO_COLOR } from './constants';
2
3
4  export const getPieceColor = (piece) => {
5    if (!piece) return null;
6    return piece === piece.toUpperCase() ? COLORS.WHITE : COLORS.BLACK;
7  };
8
9  export const getPieceType = (piece) => {
10   if (!piece) return null;
11   return piece.toLowerCase();
12 };
13
14
15 export const isValidMove = (board, from, to) => {
16   const piece = board[from.row][from.col];
17   if (!piece) return false;
18
19   const pieceType = getPieceType(piece);
20   const pieceColor = getPieceColor(piece);
21
22   // Can't capture your own piece
23   const targetPiece = board[to.row][to.col];
24   if (targetPiece && getPieceColor(targetPiece) === pieceColor) {
25     return false;
26   }
27
28   // Calculate row and column differences
29   const rowDiff = Math.abs(to.row - from.row);
30   const colDiff = Math.abs(to.col - from.col);
31
32   switch (pieceType) {
33     case PIECE_TYPES.PAWN:
34       return isValidPawnMove(board, from, to, pieceColor);
35     case PIECE_TYPES.ROOK:
36       return isValidRookMove(board, from, to);
37     case PIECE_TYPES.KNIGHT:
38       return isValidKnightMove(rowDiff, colDiff);
39     case PIECE_TYPES.BISHOP:
40       return isValidBishopMove(board, from, to);
41     case PIECE_TYPES.QUEEN:
42       return isValidQueenMove(board, from, to);
43     case PIECE_TYPES.KING:
44       return isValidKingMove(rowDiff, colDiff);
45     default:
46       return false;
47   }
48 };
49
```

Figure A.1: Deepseek React Native piece movement logic. Part 1

```

49
50 const isValidPawnMove = (board, from, to, color) => {
51   const direction = color === COLORS.WHITE ? -1 : 1;
52   const startRow = color === COLORS.WHITE ? 6 : 1;
53   const rowDiff = to.row - from.row;
54   const colDiff = Math.abs(to.col - from.col);
55
56   // Moving forward
57   if (to.col === from.col) {
58     // Single move forward
59     if (rowDiff === direction && !board[to.row][to.col]) {
60       return true;
61     }
62     // Double move from starting position
63     if (from.row === startRow && rowDiff === 2 * direction &&
64         !board[from.row + direction][from.col] &&
65         !board[to.row][to.col]) {
66       return true;
67     }
68     return false;
69   }
70
71   // Capturing diagonally
72   if (colDiff === 1 && rowDiff === direction && board[to.row][to.col]) {
73     return true;
74   }
75
76   return false;
77 };
78
79 const isValidRookMove = (board, from, to) => {
80   // Must move in straight line
81   if (from.row !== to.row && from.col !== to.col) return false;
82
83   // Check if path is clear
84   if (from.row === to.row) {
85     // Horizontal move
86     const start = Math.min(from.col, to.col) + 1;
87     const end = Math.max(from.col, to.col);
88     for (let col = start; col < end; col++) {
89       if (board[from.row][col]) return false;
90     }
91   } else {
92     // Vertical move
93     const start = Math.min(from.row, to.row) + 1;
94     const end = Math.max(from.row, to.row);
95     for (let row = start; row < end; row++) {
96       if (board[row][from.col]) return false;
97     }
98   }

```

Figure A.2: Deepseek React Native piece movement logic. Part 2

```

102
103 const isValidKnightMove = (rowDiff, colDiff) => {
104   return (rowDiff === 2 && colDiff === 1) || (rowDiff === 1 && colDiff === 2);
105 };
106
107 const isValidBishopMove = (board, from, to) => {
108   // Must move diagonally
109   if (Math.abs(from.row - to.row) !== Math.abs(from.col - to.col)) return false;
110
111   // Check if path is clear
112   const rowStep = from.row < to.row ? 1 : -1;
113   const colStep = from.col < to.col ? 1 : -1;
114   let row = from.row + rowStep;
115   let col = from.col + colStep;
116
117   while (row !== to.row && col !== to.col) {
118     if (board[row][col]) return false;
119     row += rowStep;
120     col += colStep;
121   }
122
123   return true;
124 };
125
126 const isValidQueenMove = (board, from, to) => {
127   return isValidRookMove(board, from, to) || isValidBishopMove(board, from, to);
128 };
129
130 const isValidKingMove = (rowDiff, colDiff) => {
131   return rowDiff <= 1 && colDiff <= 1;
132 };
133
134 export const movePiece = (board, from, to) => {
135   const newBoard = [...board.map(row => [...row])];
136   newBoard[to.row][to.col] = newBoard[from.row][from.col];
137   newBoard[from.row][from.col] = '';
138   return newBoard;
139 };

```

Figure A.3: Deepseek React Native piece movement logic. Part 3

```

1 package com.example.chessgame
2
3 import kotlin.math.abs
4
5 // MoveValidator.kt
6 object MoveValidator {
7     fun isValidMove(piece: ChessPiece, toPosition: Position, game: ChessGame): Boolean {
8         // Check if the position is within bounds
9         if (toPosition.x !in 0..7 || toPosition.y !in 0..7) return false
10
11         // Check if the piece is moving to its current position
12         if (piece.position == toPosition) return false
13
14         return when (piece.type) {
15             ChessPiece.PieceType.PAWN -> isValidPawnMove(piece, toPosition, game)
16             ChessPiece.PieceType.ROOK -> isValidRookMove(piece, toPosition, game)
17             ChessPiece.PieceType.KNIGHT -> isValidKnightMove(piece, toPosition, game)
18             ChessPiece.PieceType.BISHOP -> isValidBishopMove(piece, toPosition, game)
19             ChessPiece.PieceType.QUEEN -> isValidQueenMove(piece, toPosition, game)
20             ChessPiece.PieceType.KING -> isValidKingMove(piece, toPosition, game)
21         }
22     }
23
24     private fun isValidPawnMove(piece: ChessPiece, toPosition: Position, game: ChessGame): Boolean {
25         val direction = if (piece.color == ChessPiece.PieceColor.WHITE) -1 else 1
26         val startRow = if (piece.color == ChessPiece.PieceColor.WHITE) 6 else 1
27
28         // Move forward one square
29         if (toPosition.x == piece.position.x && toPosition.y == piece.position.y + direction) {
30             return game.getPieceAt(toPosition) == null
31         }
32
33         // Move forward two squares from starting position
34         if (toPosition.x == piece.position.x && piece.position.y == startRow &&
35             toPosition.y == piece.position.y + 2 * direction) {
36             val intermediatePos = Position(toPosition.x, piece.position.y + direction)
37             return game.getPieceAt(toPosition) == null && game.getPieceAt(intermediatePos) == null
38         }
39
40         // Capture diagonally
41         if (Math.abs(toPosition.x - piece.position.x) == 1 &&
42             toPosition.y == piece.position.y + direction) {
43             val targetPiece = game.getPieceAt(toPosition)
44             return targetPiece != null && targetPiece.color != piece.color
45         }
46
47         return false
48     }
49 }

```

Figure A.4: Deepseek Kotlin piece movement logic. Part 1

```

50 private fun isValidRookMove(piece: ChessPiece, toPosition: Position, game: ChessGame): Boolean {
51     // Must move in straight line
52     if (piece.position.x != toPosition.x && piece.position.y != toPosition.y) {
53         return false
54     }
55
56     // Check if path is clear
57     if (piece.position.x == toPosition.x) {
58         // Vertical move
59         val yRange = if (piece.position.y < toPosition.y) {
60             (piece.position.y + 1) until toPosition.y
61         } else {
62             (toPosition.y + 1) until piece.position.y
63         }
64
65         for (y in yRange) {
66             if (game.getPieceAt(Position(piece.position.x, y)) != null) {
67                 return false
68             }
69         }
70     } else {
71         // Horizontal move
72         val xRange = if (piece.position.x < toPosition.x) {
73             (piece.position.x + 1) until toPosition.x
74         } else {
75             (toPosition.x + 1) until piece.position.x
76         }
77
78         for (x in xRange) {
79             if (game.getPieceAt(Position(x, piece.position.y)) != null) {
80                 return false
81             }
82         }
83     }
84
85     // Check if destination is empty or has opponent's piece
86     val targetPiece = game.getPieceAt(toPosition)
87     return targetPiece == null || targetPiece.color != piece.color
88 }
89

```

Figure A.5: Deepseek Kotlin piece movement logic. Part 2

```

90 private fun isValidKnightMove(piece: ChessPiece, toPosition: Position, game: ChessGame): Boolean {
91     val dx = Math.abs(toPosition.x - piece.position.x)
92     val dy = Math.abs(toPosition.y - piece.position.y)
93
94     // Knight moves in L-shape
95     if (!((dx == 1 && dy == 2) || (dx == 2 && dy == 1))) {
96         return false
97     }
98
99     // Check if destination is empty or has opponent's piece
100     val targetPiece = game.getPieceAt(toPosition)
101     return targetPiece == null || targetPiece.color != piece.color
102 }
103
104 private fun isValidBishopMove(piece: ChessPiece, toPosition: Position, game: ChessGame): Boolean {
105     val dx = toPosition.x - piece.position.x
106     val dy = toPosition.y - piece.position.y
107
108     // Must move diagonally
109     if (Math.abs(dx) != Math.abs(dy)) {
110         return false
111     }
112
113     // Check if path is clear
114     val xStep = if (dx > 0) 1 else -1
115     val yStep = if (dy > 0) 1 else -1
116     var x = piece.position.x + xStep
117     var y = piece.position.y + yStep
118
119     while (x != toPosition.x && y != toPosition.y) {
120         if (game.getPieceAt(Position(x, y)) != null) {
121             return false
122         }
123         x += xStep
124         y += yStep
125     }
126
127     // Check if destination is empty or has opponent's piece
128     val targetPiece = game.getPieceAt(toPosition)
129     return targetPiece == null || targetPiece.color != piece.color
130 }

```

Figure A.6: Deepseek Kotlin piece movement logic. Part 3

```

131 private fun isValidQueenMove(piece: ChessPiece, toPosition: Position, game: ChessGame): Boolean {
132     // Queen combines rook and bishop moves
133     return isValidRookMove(piece, toPosition, game) || isValidBishopMove(piece, toPosition, game)
134 }
135
136 private fun isValidKingMove(piece: ChessPiece, toPosition: Position, game: ChessGame): Boolean {
137     val dx = Math.abs(toPosition.x - piece.position.x)
138     val dy = Math.abs(toPosition.y - piece.position.y)
139
140     // King moves one square in any direction
141     if (dx > 1 || dy > 1) {
142         return false
143     }
144
145     // Check if destination is empty or has opponent's piece
146     val targetPiece = game.getPieceAt(toPosition)
147     return targetPiece == null || targetPiece.color != piece.color
148 }
149 }
150

```

Figure A.7: Deepseek Kotlin piece movement logic. Part 4

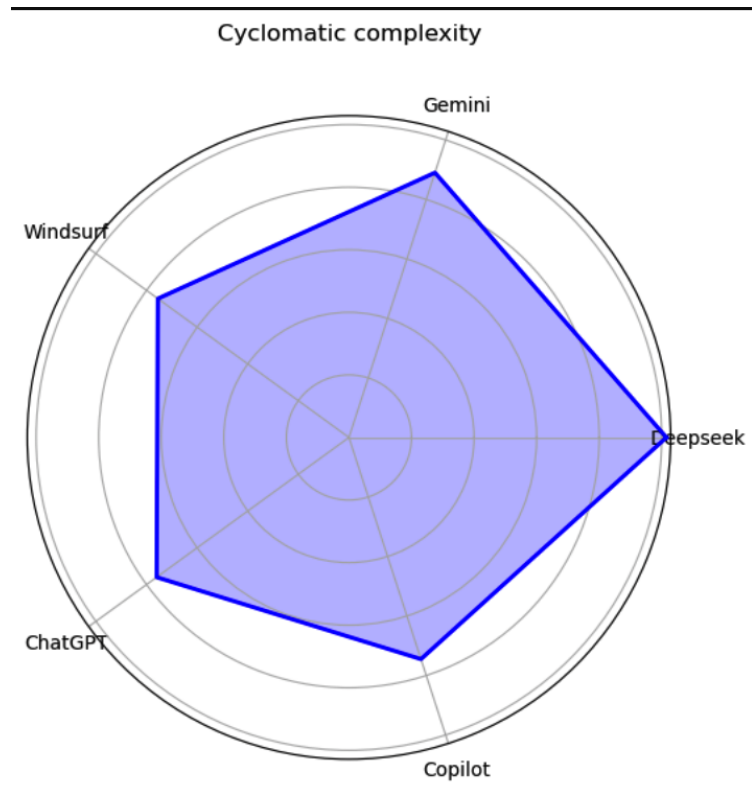


Figure A.8: Radar Chart: AI average values for Cyclomatic Complexity

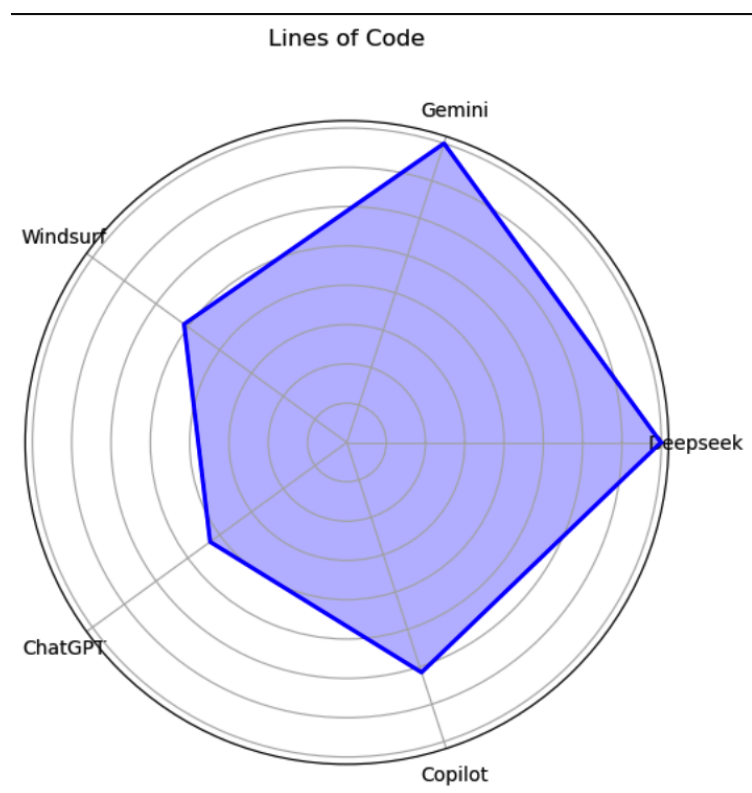


Figure A.9: Radar Chart: AI average values for Lines of Code

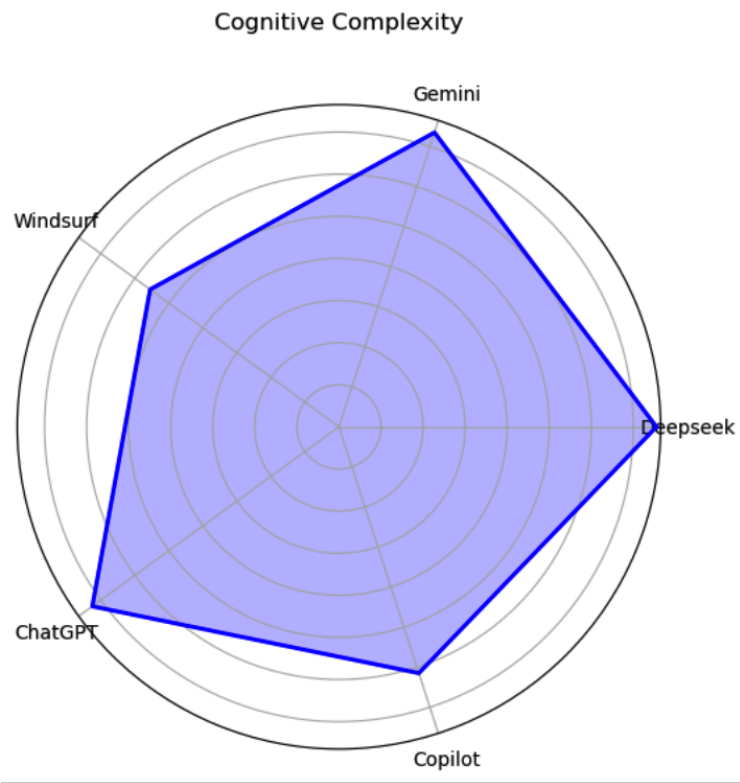


Figure A.10: Radar Chart: AI average values for Cognitive Complexity

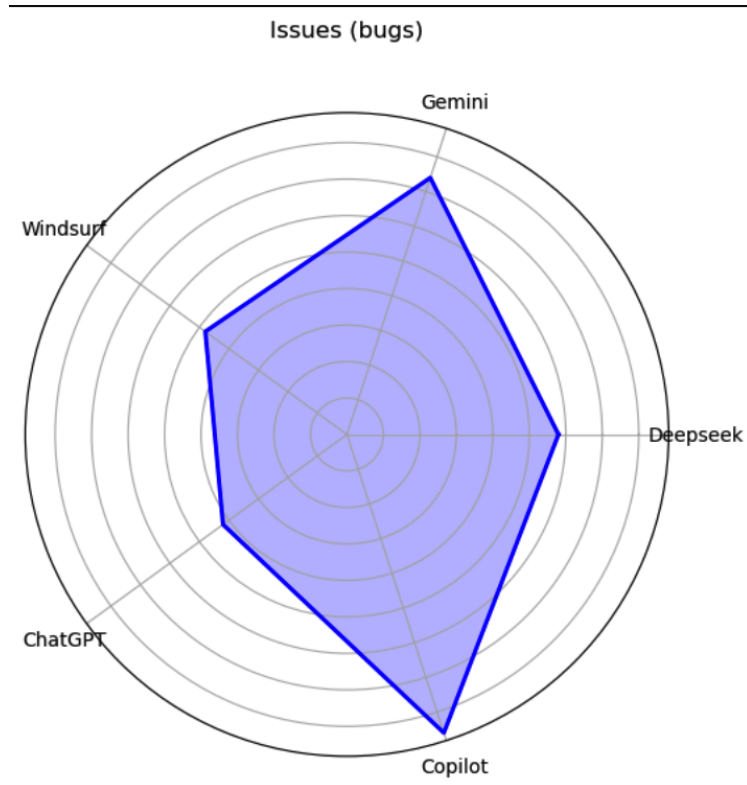


Figure A.11: Radar Chart: AI average values for Issues (bugs)

